

Client Form → CRM

Real-Estate Lead Intake & CRM Automation for Multiple Lead Sources

An 8n pipeline that captures property inquiries from more than one intake source, validates and qualifies each one against a consistent ruleset, logs every lead to CRM, and routes it to the right next action — automatically.

Project Snapshot

Industry	Real Estate Teams & Brokerages
Automation Type	Lead Intake, Qualification & CRM Routing
Built With	n8n • Typeform • Webhook • JavaScript • Notion • Gmail
Primary Objective	Turn inbound property inquiries into structured, validated, CRM-ready records and route each one to the appropriate next action.

The Business Problem

Real-estate teams rarely receive inquiries from just one place. A buyer might fill out a form embedded on the brokerage's website, or a lead might arrive through a completely different channel — a landing page, a partner tool, or another system posting data directly. Each entry point tends to produce data in its own shape, which makes consistent handling difficult from the very first step.

From there, the problems compound. Submissions get qualified inconsistently — the same inquiry can be treated as a priority by one agent and overlooked by another, because there's no shared rule for what counts as a serious buyer that gets applied every time. Incomplete submissions are common, and following up to request missing details is repetitive work that competes with an agent's time for showings and calls. Meanwhile, any inquiry that isn't manually logged into a CRM effectively disappears — no record, no audit trail, no way to revisit it. A genuinely qualified buyer can go unanswered simply because of how intake is structured, not because of anything about the lead itself.

The Solution

Client Form → CRM replaces manual intake with a single automated pipeline that treats every lead the same way, no matter where it came from. Two entry points — a Typeform trigger and a generic Webhook — both feed into one shared processing chain: Capture → Normalize → Validate → Qualify → Log → Route → Act.

Normalization brings both sources into one common shape before anything else happens, so the rest of the pipeline never has to know which channel a lead arrived through. A single validation and qualification step then checks every submission against the same rules — completeness, a valid email format, a minimum level of inquiry detail, and a budget threshold — and assigns one of three outcomes. Every lead is logged to the CRM before any communication goes out, so a record exists regardless of the qualification result. Only then does the workflow route execution to the appropriate next action: an internal alert, a client-facing response, or a request for missing details.

This automates the first operational layer of lead handling — intake, qualification, and routing. It is not a full sales platform or CRM replacement.

Workflow Architecture



Current architecture: two intake sources converge into one shared pipeline, a move-in-date check that determines the CRM-logging path, and a single routing decision with three outcomes.

Both entry points are normalized into a common structure before converging on the shared validation and qualification step, so Typeform submissions and webhook payloads are indistinguishable by the time they reach validation. Every lead then passes through the same qualification logic, a move-in-date check that determines which of two CRM-logging paths it takes, and a single routing decision. The design is intentionally linear per lead: one inquiry in, one structured CRM record and one communication outcome out — with nothing left unresolved.

How the Workflow Works

01 Capture

A Typeform Trigger and a Webhook each listen for new inquiries and fire immediately on submission, so processing begins the moment a lead arrives, regardless of source.

02 Normalize

Two dedicated Set nodes — Normalize Typeform Lead and Normalize Webhook Lead — translate each source's raw payload into one shared shape (name, email, inquiry details, budget range, move-in date, source), with fallback field-matching on the Webhook side to absorb a few different payload shapes.

03 Validate

A single JavaScript node, Validate & Qualify Lead, checks every field for presence, validates the email format, confirms the inquiry description meets a minimum length, and checks the move-in date against a real calendar rather than just a text pattern.

04 Qualify

The same node parses the stated budget — ranges, "k"/"m" shorthand, currency symbols, and ceiling language like "under \$500k" — into structured minimum, maximum, and value fields, then assigns Priority Buyer, Standard Buyer, or Missing Info against the \$750,000 threshold.

05 Handle Move-in Date

A conditional check (Has Move-in Date?) determines whether a valid move-in date was provided for this submission.

06 Log to CRM

The lead is written to the Notion CRM through one of two dedicated nodes — Log Lead to CRM (with the move-in date) or Log Lead to CRM — No Date (without it) — so a missing or invalid date does not prevent the lead from being logged through the fallback CRM path.

07 Prepare Routed Lead

A bridging Set node reunifies both CRM paths into one consistent object and attaches the newly created CRM page's ID and URL, so everything downstream is identical regardless of which logging path ran.

08 Route

A rules-based Switch node, Route Lead, reads the qualification result and directs execution to exactly one of three named outputs.

09 Act

A dedicated Gmail node sends the corresponding message: an internal alert, a client-facing response, or a request for missing details.

Qualification Logic

Priority Buyer Budget minimum ≥ \$750,000 Complete, valid submission with a stated minimum budget at or above the priority threshold.	Standard Buyer Budget minimum < \$750,000 Complete, valid submission with a stated minimum budget below the priority threshold.	Missing Info Required data absent or invalid Name, email format, inquiry length, budget, or move-in date failed validation.
---	--	---

Every field is checked the same way for every lead: name and email presence, email format, a minimum inquiry length, budget presence, and move-in date validity. Anything that fails is recorded explicitly in a missingFields list rather than silently dropped, and budget parsing is flexible enough to handle ranges, "k"/"m" shorthand, and qualifying language so the \$750,000 threshold is applied consistently no matter how a prospect phrases their number. Qualification is entirely rule-based — there is no AI or LLM involved in the current workflow.

CRM Logging & Routing Outcomes

Every lead is written to the Notion CRM before any email goes out — qualified or not. Because a move-in date isn't always provided or valid, the workflow uses two separate CRM-logging paths rather than forcing every record through the same field requirements: one logs the lead with its move-in date attached, the other logs an equivalent record without it. Either way, the lead exists in the CRM immediately, with nothing dependent on a downstream email actually sending.

Priority Buyer → Internal priority-alert emailed to the business owner, with the buyer overview and a direct CRM link.

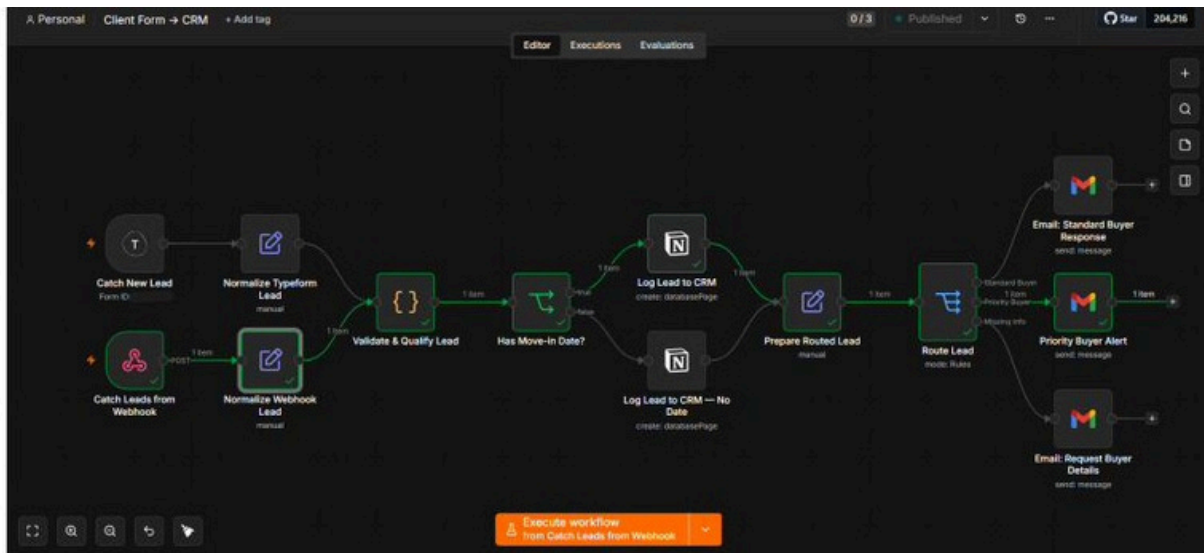
Standard Buyer → Client-facing acknowledgment that states the current priority-buyer threshold explicitly.

Missing Info → Client-facing request listing the specific missing details for that submission.

Demonstrated Workflow Evidence

The following is drawn directly from the current, active workflow — not mockups or renders. Names and emails are synthetic test data; no real client information was used.

1. The live workflow canvas



The current n8n canvas: two entry points, one shared qualification pipeline, two conditional CRM-logging paths, and three terminal email outcomes — matching the architecture diagram node-for-node. The Typeform trigger and its own normalization node are visible as a first-class entrypoint alongside the Webhook.

2. A submission before and after qualification — Rachel Kim

```
## Lead Input
---json
{
  "name": "Rachel Kim",
  "email": "rachel.kim.buyer@example.com",
  "inquiryDetails": "Relocating for a new job and looking for a move-in-ready 4-bedroom home in the Cherry Creek area of Denver. Would like to begin scheduling private showings as soon as possible.",
  "budgetRange": "$750,000 - $900,000",
  "moveInDate": "2026-11-01",
  "source": "webhook"
}
```

```
## Lead Output

'''json
{
  "name": "Rachel Kim",
  "email": "rachel.kim.buyer@example.com",
  "emailValid": true,
  "inquiryDetails": "Relocating for a new job and looking for a move-in-ready 4-bedroom home in the Cherry Creek area of Denver. Would like to begin scheduling private showings as soon as possible.",
  "budgetRange": "$750,000 - $900,000",
  "budgetMin": 750000,
  "budgetMax": 900000,
  "budgetValue": 750000,
  "moveInDate": "2026-11-01",
  "source": "webhook",
  "leadPriority": "Priority Buyer",
  "missingFields": [],
  "qualifiedAt": "2026-09-14T05:56:14.944Z",
  "crmPageId": " ",
  "crmUrl": "https://app.notion.com/p/Rachel-Kim"
}
'''
```

Rachel Kim's inquiry, entering through the Webhook path, before and after the Validate & Qualify Lead step. The output shows the derived budgetMin / budgetMax / budgetValue, an empty missingFields list, and a Priority Buyer result — plus the CRM page ID and URL that get attached two steps later.

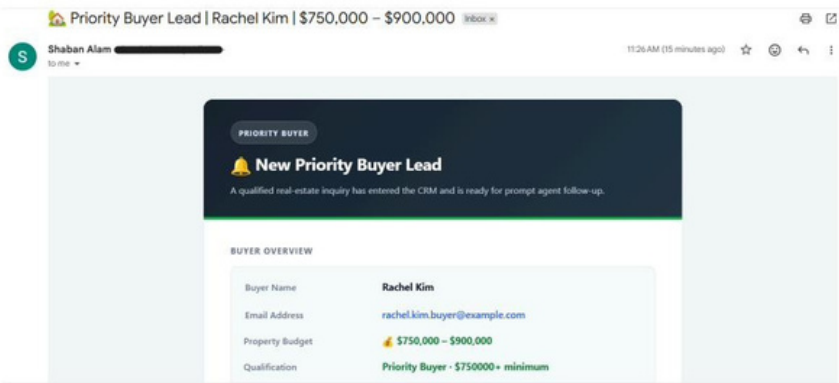
3. The resulting CRM record

The screenshot shows a Notion database titled "CRM Log Lead". It contains a table with the following columns: As Name, Email, Inquiry Details, Budget Value, Status, Move-In Date, and Budget Range. Three records are visible:

As Name	Email	Inquiry Details	Budget Value	Status	Move-In Date	Budget Range
Rachel Kim	rachel.kim.buyer@example.com	Relocating for a new job and lo	750000	Priority Buyer	November 1, 2026 12:00 AM	750000 - 900000
Marcus Webb	marcus.webb.renter@example	Looking to rent a 1-bedroom ap	1200	Standard Buyer	October 15, 2026 12:00 AM	1200 - 1500
Grace Whitfield	grace.whitfield.buyer@example	Interested in a 4-bedroom hom	650000	Missing Info		\$650,000 - \$700,000

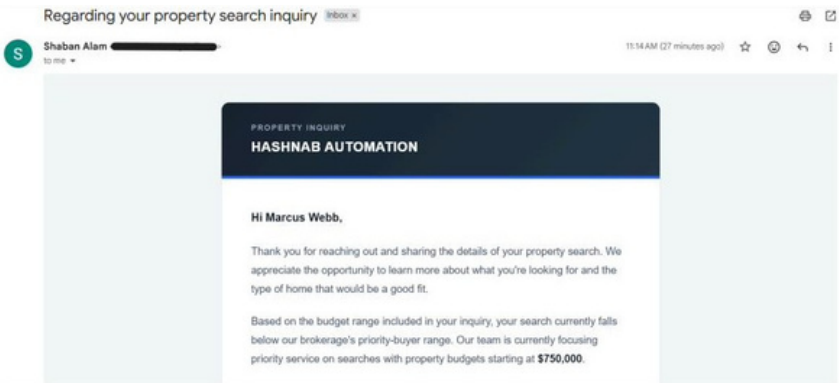
The live CRM Log Lead database after three test submissions. Rachel Kim (Priority Buyer) and Marcus Webb (Standard Buyer) both have a valid move-in date and a Budget Range stored as plain numbers — the dated CRM-logging path. Grace Whitfield (Missing Info) has no move-in date, and her Budget Range is instead stored as the raw submitted string — the fallback path. This is direct evidence that the two CRM nodes are doing different, intentional jobs rather than duplicating each other.

4. Priority Buyer — internal alert



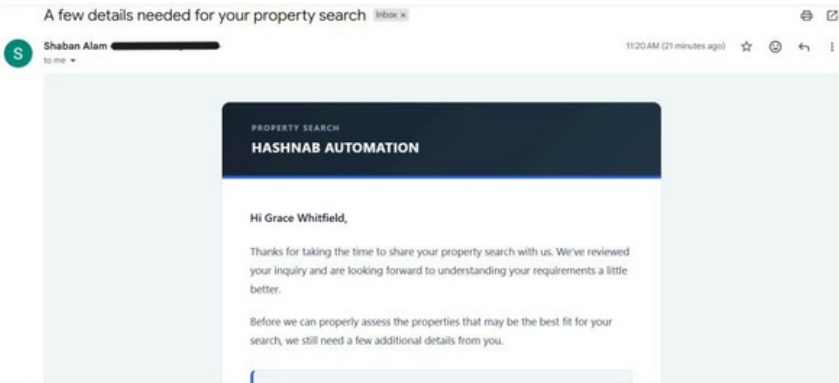
The internal alert triggered by Rachel Kim's submission. This goes to the business owner, not the prospect, and carries the qualification basis, stated budget, and (below the visible crop) a direct link to the CRM record.

5. Standard Buyer — client-facing response



The client-facing response sent to Marcus Webb, whose stated budget fell below the \$750,000 threshold. The email states the threshold explicitly rather than leaving the prospect without an explanation.

6. Missing Info — client-facing follow-up



The follow-up sent to Grace Whitfield. The requested details are generated from her specific missingFields list rather than a generic template.

Business Impact

This is a demonstration build, not a live client deployment — so there's no ROI or conversion data to report here. What it does show is a system designed to help a real-estate team:

- Standardize inbound handling across every entry point, so response quality doesn't depend on which agent happens to pick up a lead
- Remove repetitive manual qualification and CRM data entry
- Keep every inquiry centralized and recorded, including incomplete ones
- Surface high-value buyers immediately rather than mixed in with everything else
- Respond to every inquiry — priority, standard, or incomplete — so nothing goes unanswered

Technical Highlights

- Multiple intake sources (Typeform + Webhook) sharing one normalization layer
- Deterministic, rule-based qualification — no AI or LLM involved
- Real email-format validation, not just presence checking
- Flexible budget parsing: ranges, k/m shorthand, currency symbols, and ceiling language
- Explicit missing-field tracking (missingFields) instead of silent failure
- Calendar-aware move-in-date validation that rejects invalid dates, not just malformed ones
- Conditional CRM logging that preserves the a record exists even without a valid date
- Rules-based routing with CRM deep-link propagation into the internal alert
- Modular, single-responsibility nodes — swapping the CRM or form provider touches one node, not the pipeline

Current Scope

Typeform + Webhook intake, JavaScript-based validation and qualification, Notion CRM, Gmail-based outcomes, a \$750,000 Priority Buyer threshold, and fully rule-based (non-AI) qualification logic. This is the current, demonstrated scope of the system.

Potential Extensions

Not part of the current build — directions the same architecture could grow into:

- AI-assisted qualification for richer signal beyond threshold-based rules
- Direct CRM integrations beyond Notion (HubSpot, Salesforce, Follow Up Boss)
- Lead deduplication against existing CRM records
- Slack or Telegram alerts alongside email
- Scheduling links embedded directly in the priority-buyer alert
- Automated follow-up sequences for non-responders

Let's Build Yours

Have a lead-intake process that still runs on manual review and copy-paste CRM entry? I design automations like this one around a business's existing tools and rules, not a one-size-fits-all template. If you want to talk through what that could look like for your team, reach out.

Shaban Alam

PythonAutomation Developer · n8n Workflow Specialist

shabandev27@gmail.com

github.com/Shaban27-dev